Table des matières

Labo 09 OLTP — Séries temporelles et as-of joins sur le transactionnel 1
Le terrain, en une minute 1
Partie 1 — Le grain temporel, avant tout le reste 2
Partie 2 — La série quotidienne et l'axe complet 4
Partie 3 — Les cadres en temps 6
Partie 4 — L'as-of join, le cœur du module 7
Partie 5 — La marge réelle, deux as-of joins d'un coup 11
Partie 6 — Sessionisation et cohortes 12
Partie 7 — Ce que le contraste vous a montré 14
Nettoyage 15
Et ensuite ? 15

Labo 09 OLTP — Séries temporelles et as-of joins sur le transactionnel

Laboratoire du Cours 09, voie OLTP · PostgreSQL + AdventureWorks OLTP · Formation SQL avancé

On refait le laboratoire OLAP, partie pour partie, sur la base transactionnelle `Adventureworks` dans PostgreSQL. Connectez-vous avec pgAdmin, DBeaver ou `psql`. Comptez une heure et demie.

Les deux voies — Ce labo est le jumeau de `Labo_09_OLAP_SQL.md`. La différence n'est pas seulement syntaxique cette fois : l'historique des prix vit ici dans une table dédiée, propre et bien tenue, alors que l'entrepôt le replie dans sa dimension avec des dates cassées. Vous verrez donc le mécanisme de l'as-of join sans bruit, et vous comprendrez mieux pourquoi la règle de la section 6 de la théorie insiste sur la date de début.

Ce que ce labo démontre chiffres en main — Deux écritures naïves de l'as-of join perdent respectivement 2 898 et 85 696 lignes sur les 121 317 de `sales.salesorderdetail`, sans lever la moindre erreur. Vous allez les écrire, compter, et comprendre.

Le terrain, en une minute

© 2026 iAlexMG. Tous droits réservés.

Table	Ce qu'elle contient	Colonnes utiles
`sales.salesorderheader`	une ligne par commande, 31 465 lignes	`salesorderid`, `customerid`, `orderdate`, `totaldue`
`sales.salesorderdetail`	une ligne par ligne de commande, 121 317 lignes	`salesorderid`, `productid`, `orderqty`, `unitprice`, `linetotal`
`production.product`	le catalogue, 504 produits	`productid`, `name`, `listprice`, `standardcost`
`production.productlistpricehistory`	l'historique du prix de liste, 395 lignes	`productid`, `startdate`, `enddate`, `listprice`
`production.productcosthistory`	l'historique du coût standard, 395 lignes	`productid`, `startdate`, `enddate`, `standardcost`

Les deux tables d'historique sont la matière du module. Notez qu'elles existent : dans le monde transactionnel, l'historique d'un attribut est une table à part, que l'application alimente.

On travaillera dans un schéma dédié, facile à supprimer à la fin.

```sql
CREATE SCHEMA IF NOT EXISTS labo09;
SET search_path TO labo09, public;
```

Partie 1 — Le grain temporel, avant tout le reste

1.1 Les bornes réelles

Théorie : `Theorie_09_SQL.md`, section 1.

```sql
SELECT MIN(orderdate) AS premiere, MAX(orderdate) AS derniere, COUNT(*) AS commandes
FROM   sales.salesorderheader;
```

Attendu : du 30 mai 2022 au 29 juin 2025, 31 465 commandes. Retenez ces bornes, elles servent partout dans ce labo.

1.2 Le piège de l'intervalle fermé

Théorie : `Theorie_09_SQL.md`, section 1.

```sql
-- Version fautive : BETWEEN sur une colonne horodatée.
SELECT COUNT(*) AS avec_between
FROM   sales.salesorderheader
WHERE  orderdate BETWEEN '2023-01-01' AND '2023-12-31';
```


© 2026 iAlexMG. Tous droits réservés.

```sql
-- Version correcte : semi-ouvert.
SELECT COUNT(*) AS semi_ouvert
FROM   sales.salesorderheader
WHERE  orderdate >= '2023-01-01' AND orderdate < '2024-01-01';
```

Sur ce portage, les commandes sont horodatées à minuit, donc les deux comptes coïncident. Ne concluez surtout pas que le piège n'existe pas : il dort. Le jour où l'application se met à enregistrer l'heure réelle, la première requête perd une journée par an et personne ne s'en aperçoit, parce qu'elle continue de renvoyer un nombre plausible. Prenez l'habitude maintenant.

1.3 Ce que dit l'historique des prix

Théorie : `Theorie_09_SQL.md`, sections 1 et 6.

```sql
SELECT COUNT(*)                     AS versions,
       COUNT(DISTINCT productid)    AS produits,
       MIN(startdate)               AS premiere_version,
       MAX(startdate)               AS derniere_version,
       COUNT(*) FILTER (WHERE enddate IS NULL) AS versions_ouvertes
FROM   production.productlistpricehistory;
```

Attendu : 395 versions, 293 produits, des débuts au 30 mai 2022 et au 29 mai 2024, et 195 versions sans date de fin. Cette dernière colonne est celle qui compte pour la suite : une version sans date de fin est la version courante, celle qui est encore en vigueur.

```sql
-- Les produits qui ont changé de prix au moins une fois.
SELECT COUNT(*) AS produits_a_plusieurs_versions
FROM   (SELECT productid FROM production.productlistpricehistory
        GROUP BY productid HAVING COUNT(*) > 1) AS t;
```

Attendu : 77. Regardez-en un de près :

```sql
SELECT * FROM production.productlistpricehistory
WHERE  productid = 707 ORDER BY startdate;
```

Trois versions, dont les deux premières portent le même prix. Un historique enregistre des périodes de validité, pas seulement des changements de valeur : c'est une distinction qui compte quand on compte des lignes.

1.4 Les journées qui ne durent pas 24 heures

Théorie : `Theorie_09_SQL.md`, section 1.

La même question que sur l'entrepôt, les heures du 1er novembre 2026 à Montréal :

```sql
SELECT COUNT(*) AS heures_du_1er_novembre
FROM   generate_series(TIMESTAMP '2026-11-01 00:00:00' AT TIME ZONE 'America/Toronto',
```


© 2026 iAlexMG. Tous droits réservés.

```
                    TIMESTAMP '2026-11-02 00:00:00' AT TIME ZONE 'America/Toronto' - INTERVAL '1 hour',
                    INTERVAL '1 hour') AS g(h);
```

Attendu : 25, comme sur DuckDB. La durée de la journée, elle, ne sort pas pareil :

```
SELECT (TIMESTAMP '2026-11-02 00:00:00' AT TIME ZONE 'America/Toronto')
     - (TIMESTAMP '2026-11-01 00:00:00' AT TIME ZONE 'America/Toronto') AS soustraction,
       TIMESTAMPTZ '2026-11-01 00:00:00-04' + INTERVAL '1 day'    AS plus_un_jour,
       TIMESTAMPTZ '2026-11-01 00:00:00-04' + INTERVAL '24 hours' AS plus_24_heures;
```

Attendu : `1 day 01:00:00`, là où l'entrepôt rendait `1 day`. Puis le 2 novembre à minuit d'un côté, le 1er novembre à 23 h de l'autre : un jour de calendrier et 24 heures ne sont pas la même durée cette nuit-là, et les deux moteurs le savent. Ce qu'ils ne font pas pareil, c'est la soustraction.

Partie 2 — La série quotidienne et l'axe complet

2.1 La série brute

Théorie : `Theorie_09_SQL.md`, section 2.

```
CREATE TABLE labo09.ventes_jour AS
SELECT h.orderdate::date       AS jour,
       SUM(d.linetotal)        AS ventes,
       COUNT(DISTINCT h.salesorderid) AS commandes
FROM   sales.salesorderheader AS h
JOIN   sales.salesorderdetail AS d ON d.salesorderid = h.salesorderid
GROUP  BY 1;

SELECT COUNT(*) AS jours_actifs FROM labo09.ventes_jour;
```

Notez ce compte. Comparez-le au nombre de jours de la plage :

```
SELECT MAX(jour) - MIN(jour) + 1 AS jours_de_la_plage FROM labo09.ventes_jour;
```

L'écart entre les deux est le nombre de jours sans aucune vente. Ils n'existent pas dans la table, et rien dans un graphique ne le dirait.

2.2 L'axe complet

Théorie : `Theorie_09_SQL.md`, section 2.

```
CREATE TABLE labo09.serie AS
WITH bornes AS (SELECT MIN(jour) AS d1, MAX(jour) AS d2 FROM labo09.ventes_jour),
axe AS (
```


© 2026 iAlexMG. Tous droits réservés.

```sql
    SELECT g::date AS jour
    FROM   bornes, generate_series(bornes.d1, bornes.d2, INTERVAL '1 day') AS g
)
SELECT a.jour,
       COALESCE(v.ventes, 0)    AS ventes,
       COALESCE(v.commandes, 0) AS commandes,
       (v.jour IS NULL)         AS jour_vide
FROM   axe AS a
LEFT   JOIN labo09.ventes_jour AS v ON v.jour = a.jour;

SELECT COUNT(*) AS jours_axe, COUNT(*) FILTER (WHERE jour_vide) AS jours_vides
FROM   labo09.serie;
```

Vous avez maintenant une série sans trou. La colonne `jour_vide` garde la trace de ce qui a été inventé, ce qui est une bonne habitude : un zéro reconstitué et un zéro observé ne se valent pas.

2.3 Les deux moyennes

Théorie : `Theorie_09_SQL.md`, sections 2 et 4.

```sql
SELECT AVG(ventes) AS moyenne_sur_axe_complet FROM labo09.serie;
SELECT AVG(ventes) AS moyenne_sur_jours_actifs FROM labo09.ventes_jour;
```

Les deux nombres diffèrent. Aucun n'est faux, ils répondent à deux questions distinctes. Ce qui serait faux, c'est de publier l'un en croyant avoir calculé l'autre.

2.4 Des périodes de largeur libre

Théorie : `Theorie_09_SQL.md`, section 5.

PostgreSQL n'a pas `TIME_BUCKET` ; il a `DATE_BIN`, qui demande son origine. Les commandes par quinzaine, sur les deux premiers mois de 2025, comptées à partir du lundi 3 janvier 2000, l'origine que DuckDB prend d'office :

```sql
SELECT DATE_BIN(INTERVAL '14 days', orderdate, TIMESTAMP '2000-01-03') AS quinzaine,
       COUNT(*)                                                        AS commandes,
       ROUND(SUM(subtotal), 2)                                         AS ventes
FROM   sales.salesorderheader
WHERE  orderdate >= '2025-01-01' AND orderdate < '2025-03-01'
GROUP  BY 1
ORDER  BY 1;
```

Attendu : cinq quinzaines, du lundi 30 décembre 2024 au lundi 24 février 2025, avec 753, 911, 1 065, 864 et 400 commandes. Écrivez la même requête avec l'origine `TIMESTAMP '2000-01-01'` : les quinzaines commencent un samedi, le 28 décembre 2024, et comptent 629, 894, 1 067, 870 et 533 commandes. La même question, deux découpages, et rien ne dit lequel est le bon sinon l'origine écrite.

© 2026 iAlexMG. Tous droits réservés.

Erreur voulue : une largeur en mois.

```sql
SELECT DATE_BIN(INTERVAL '1 month', orderdate, TIMESTAMP '2000-01-01') AS mois
FROM   sales.salesorderheader
LIMIT  1;
```

Attendu : une erreur, « Les données de type timestamp ne peuvent pas être rangées dans des intervalles contenant des mois ou des années ». Un mois n'a pas de largeur fixe ; pour le mois, c'est `DATE_TRUNC('month', …)` qui sert, sur les deux terrains.

Partie 3 — Les cadres en temps

3.1 La moyenne mobile en lignes et en durée

Théorie : `Theorie_09_SQL.md`, section 4.

```sql
SELECT jour,
       ventes,
       AVG(ventes) OVER (ORDER BY jour
                         ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) AS mm7_lignes,
       AVG(ventes) OVER (ORDER BY jour
                         RANGE BETWEEN INTERVAL '6 days' PRECEDING
                                   AND CURRENT ROW)                AS mm7_duree
FROM   labo09.serie
ORDER  BY jour
LIMIT  30;
```

Sur l'axe complet, les deux colonnes sont identiques, aux premières lignes près. C'est le résultat attendu, et c'est justement ce qu'il faut avoir vu une fois : sur une série sans trou, une ligne égale un jour, donc les deux cadres décrivent la même chose.

3.2 La même chose sur la série trouée

Théorie : `Theorie_09_SQL.md`, section 4.

```sql
SELECT jour,
       ventes,
       AVG(ventes) OVER (ORDER BY jour
                         ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) AS mm7_lignes,
       AVG(ventes) OVER (ORDER BY jour
                         RANGE BETWEEN INTERVAL '6 days' PRECEDING
                                   AND CURRENT ROW)                AS mm7_duree
FROM   labo09.ventes_jour
```


© 2026 iAlexMG. Tous droits réservés.

```sql
ORDER  BY jour
LIMIT  30;
```

Cette fois les deux colonnes divergent, parce que la table ne contient que les jours actifs. Le cadre en lignes remonte sept observations, donc parfois deux ou trois semaines. Le cadre en durée remonte sept jours et n'y trouve que ce qui existe.

Mesurez l'écart plutôt que de le regarder :

```sql
SELECT COUNT(*) AS jours,
       COUNT(*) FILTER (WHERE mm7_lignes <> mm7_duree) AS jours_ou_ca_diverge
FROM (
  SELECT jour,
         AVG(ventes) OVER (ORDER BY jour ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) AS mm7_lignes,
         AVG(ventes) OVER (ORDER BY jour RANGE BETWEEN INTERVAL '6 days' PRECEDING
                                                  AND CURRENT ROW)                  AS mm7_duree
  FROM   labo09.ventes_jour
) AS t;
```

Portabilité — L'écriture `RANGE BETWEEN INTERVAL … PRECEDING` que vous venez d'employer passe aussi sur l'entrepôt, mesuré des deux côtés. Elle n'est pourtant pas universelle, et le cadre glissant en durée est plus restreint sur d'autres moteurs, dont Snowflake d'après sa documentation. Le laboratoire jumeau passe donc par l'axe complet et `ROWS`, qui est la parade universelle : sur une série sans trou, les deux cadres coïncident par construction.

3.3 Le troisième mode, `GROUPS`

Théorie : `Theorie_09_SQL.md`, section 4.

```sql
SELECT jour, ventes,
       AVG(ventes) OVER (ORDER BY jour
                         GROUPS BETWEEN 6 PRECEDING AND CURRENT ROW) AS mm7_groupes
FROM   labo09.ventes_jour
ORDER  BY jour LIMIT 10;
```

`GROUPS` compte les groupes de valeurs d'ordre égales plutôt que les lignes. Sur une série où chaque jour n'a qu'une ligne, il se confond avec `ROWS`. Sur une table de lignes de commande, où une même date porte des dizaines de lignes, il se distingue nettement des deux autres. Retenez qu'il existe : il répond à « les six dates précédentes », ce qu'aucun des deux autres ne dit.

Partie 4 — L'as-of join, le cœur du module

4.1 La question

Théorie : `Theorie_09_SQL.md`, section 6.

© 2026 iAlexMG. Tous droits réservés.

Quel était le prix de liste en vigueur le jour où la commande a été passée ? La table `production.product` porte le prix d'aujourd'hui, pas celui de ce jour-là. Employer `p.listprice` donnerait un rapport de marge qui change chaque fois qu'on le rejoue.

4.2 La première écriture naïve, et sa facture

Théorie : `Theorie_09_SQL.md`, section 6.

```
-- Écriture fautive : l'intervalle fermé.
SELECT COUNT(*) AS lignes_appariees
FROM   sales.salesorderheader AS h
JOIN   sales.salesorderdetail AS d ON d.salesorderid = h.salesorderid
JOIN   production.productlistpricehistory AS pl
       ON pl.productid = d.productid
      AND h.orderdate BETWEEN pl.startdate AND pl.enddate;
```

Attendu : 35 621 lignes appariées sur 121 317. Vous venez de perdre 85 696 lignes, soit sept sur dix, et la requête n'a rien signalé.

La cause est la version courante : 195 des 395 versions ont `enddate` à `NULL`, et une comparaison avec `NULL` ne vaut ni vrai ni faux. Toute commande postérieure au dernier changement de prix disparaît donc, c'est-à-dire la partie la plus récente et la plus intéressante des données. C'est le `NULL` du Cours 01 qui ressort, quatre cours plus tard, sous un nouveau costume.

Vérifiez le compte total pour être sûr de la perte :

```
SELECT COUNT(*) AS lignes_totales
FROM   sales.salesorderheader AS h
JOIN   sales.salesorderdetail AS d ON d.salesorderid = h.salesorderid;
```

4.3 Le rustinage qui ne règle rien

Théorie : `Theorie_09_SQL.md`, section 6.

```
SELECT COUNT(*) AS lignes_appariees
FROM   sales.salesorderheader AS h
JOIN   sales.salesorderdetail AS d ON d.salesorderid = h.salesorderid
JOIN   production.productlistpricehistory AS pl
       ON pl.productid = d.productid
      AND h.orderdate >= pl.startdate
      AND (pl.enddate IS NULL OR h.orderdate <= pl.enddate);
```

Le compte remonte, et l'écriture reste fragile. Elle suppose que les intervalles ne se chevauchent pas et ne laissent pas de trou, ce que rien ne garantit. Ici, la table est propre et l'écriture fonctionne. Sur la dimension de l'entrepôt, elle s'effondrerait, et vous le verrez au labo jumeau.

4.4 L'écriture canonique

© 2026 iAlexMG. Tous droits réservés.

Théorie : `Theorie_09_SQL.md`, section 6.

```
CREATE TABLE labo09.ligne_prix AS
SELECT h.salesorderid,
       h.orderdate::date  AS jour,
       h.customerid,
       d.productid,
       d.orderqty,
       d.unitprice,
       d.linetotal,
       prix.listprice     AS prix_en_vigueur
FROM   sales.salesorderheader AS h
JOIN   sales.salesorderdetail AS d ON d.salesorderid = h.salesorderid
LEFT   JOIN LATERAL (
         SELECT pl.listprice
         FROM   production.productlistpricehistory AS pl
         WHERE  pl.productid  = d.productid
           AND  pl.startdate <= h.orderdate
         ORDER  BY pl.startdate DESC
         LIMIT  1
       ) AS prix ON true;

SELECT COUNT(*) AS lignes,
       COUNT(prix_en_vigueur) AS avec_prix,
       COUNT(*) - COUNT(prix_en_vigueur) AS sans_prix
FROM   labo09.ligne_prix;
```

Attendu : 121 317 lignes, toutes avec un prix, aucune sans. Rien n'a été perdu, et la date de fin n'a jamais été lue.

4.5 Le détail d'un caractère

Théorie : `Theorie_09_SQL.md`, section 6.

Remplacez <= par < dans la sous-requête et recomptez :

```
SELECT COUNT(*) AS lignes,
       COUNT(prix.listprice) AS avec_prix,
       COUNT(*) - COUNT(prix.listprice) AS sans_prix
FROM   sales.salesorderheader AS h
JOIN   sales.salesorderdetail AS d ON d.salesorderid = h.salesorderid
LEFT   JOIN LATERAL (
         SELECT pl.listprice
         FROM   production.productlistpricehistory AS pl
```


© 2026 iAlexMG. Tous droits réservés.

```
        WHERE  pl.productid = d.productid
          AND  pl.startdate < h.orderdate          -- strict au lieu de <=
        ORDER  BY pl.startdate DESC
        LIMIT  1
      ) AS prix ON true;
```

Attendu : 2 898 lignes sans prix. Ce sont les commandes passées le jour même de l'entrée en vigueur de la toute première version du prix.

Ce compte ne voit pourtant que la moitié visible du dégât. Une commande passée le jour d'un changement de prix qui n'est pas le premier ne perd pas son prix : elle reçoit l'ancien, et aucun `NULL` ne le trahit. Comparez les deux écritures ligne à ligne :

```
SELECT COUNT(*) FILTER (WHERE sans.listprice IS NULL)           AS sans_prix,
       COUNT(*) FILTER (WHERE sans.listprice <> avec.listprice) AS prix_change
FROM   sales.salesorderheader AS h
JOIN   sales.salesorderdetail AS d ON d.salesorderid = h.salesorderid
LEFT   JOIN LATERAL (
         SELECT pl.listprice
         FROM   production.productlistpricehistory AS pl
         WHERE  pl.productid = d.productid
           AND  pl.startdate <= h.orderdate
         ORDER  BY pl.startdate DESC
         LIMIT  1
       ) AS avec ON true
LEFT   JOIN LATERAL (
         SELECT pl.listprice
         FROM   production.productlistpricehistory AS pl
         WHERE  pl.productid = d.productid
           AND  pl.startdate < h.orderdate
         ORDER  BY pl.startdate DESC
         LIMIT  1
       ) AS sans ON true;
```

Attendu : 2 898 lignes sans prix, et 1 191 autres dont le prix a changé en silence. Ces 1 191 lignes sont les plus dangereuses du labo : elles ont une valeur, plausible, fausse, et rien dans un compte de `NULL` ne les signale. C'est exactement la faute silencieuse que le module nomme. Un caractère, plus de quatre mille lignes fausses, et aucune erreur. La convention « une valeur qui entre en vigueur ce jour-là s'applique ce jour-là » doit être écrite quelque part, pas devinée.

4.6 Contrôler l'historique avant de s'en servir

Théorie : `Theorie_09_SQL.md`, sections 1 et 6.

C'est le réflexe que le labo jumeau va rendre indispensable. Prenez-le ici, où il ne trouvera rien.

© 2026 iAlexMG. Tous droits réservés.

```sql
-- Des intervalles qui se chevauchent pour un même produit ?
SELECT COUNT(*) AS chevauchements
FROM   production.productlistpricehistory AS a
JOIN   production.productlistpricehistory AS b
       ON b.productid = a.productid
      AND b.startdate > a.startdate
      AND b.startdate <= COALESCE(a.enddate, DATE '9999-12-31');

-- Des fins antérieures à leur début ?
SELECT COUNT(*) AS dates_incoherentes
FROM   production.productlistpricehistory
WHERE  enddate IS NOT NULL AND enddate < startdate;

-- Plusieurs versions ouvertes pour un même produit ?
SELECT productid, COUNT(*) AS ouvertes
FROM   production.productlistpricehistory
WHERE  enddate IS NULL
GROUP  BY productid HAVING COUNT(*) > 1;
```

Attendu : zéro, zéro, aucune ligne. La table est saine. Gardez ces trois requêtes : ce sont elles qui vous diront, sur une autre base, si vous pouvez faire confiance à un intervalle de validité.

Partie 5 — La marge réelle, deux as-of joins d'un coup

Théorie : `Theorie_09_SQL.md`, section 6.

Le prix seul ne suffit pas à une marge : il faut aussi le coût du moment. Et le coût a lui aussi son historique.

```sql
SELECT lp.jour,
       lp.productid,
       lp.orderqty,
       lp.unitprice,
       lp.prix_en_vigueur,
       cout.standardcost                                  AS cout_en_vigueur,
       (lp.unitprice - cout.standardcost) * lp.orderqty   AS marge,
       ROUND(100 * (lp.unitprice - cout.standardcost)
             / NULLIF(lp.unitprice, 0), 1)                AS marge_pct
FROM   labo09.ligne_prix AS lp
LEFT   JOIN LATERAL (
         SELECT pc.standardcost
```


© 2026 iAlexMG. Tous droits réservés.

```
          FROM   production.productcosthistory AS pc
          WHERE  pc.productid  = lp.productid
            AND  pc.startdate <= lp.jour
          ORDER  BY pc.startdate DESC
          LIMIT  1
        ) AS cout ON true
  ORDER  BY lp.jour
  LIMIT  20;
```

Deux remarques. Le `NULLIF` protège la division, comme à la section 3 de la théorie. Et comparez la marge ainsi obtenue avec celle qu'on aurait calculée sur `production.product.standardcost`, le coût d'aujourd'hui : sur les commandes de 2022, l'écart est celui de trois années d'inflation des coûts, attribué rétroactivement à un exercice clos.

```
  -- La marge « anachronique », pour la comparaison.
  SELECT ROUND(AVG(100 * (lp.unitprice - p.standardcost) / NULLIF(lp.unitprice, 0)), 2) AS marge_pct_aujourdhui
  FROM   labo09.ligne_prix AS lp
  JOIN   production.product AS p ON p.productid = lp.productid
  WHERE  lp.jour < '2023-01-01';
```

C'est le genre de chiffre qui passe une revue et qui ne devrait pas.

Partie 6 — Sessionisation et cohortes

6.1 Les sessions d'achat d'un client

Théorie : `Theorie_09_SQL.md`, section 7.

```
  WITH marque AS (
      SELECT customerid,
             orderdate::date AS jour,
             salesorderid,
             CASE WHEN orderdate::date
                       - LAG(orderdate::date) OVER (PARTITION BY customerid
                                                    ORDER BY orderdate) > 120
                  THEN 1 ELSE 0 END AS rupture
      FROM   sales.salesorderheader
  )
  SELECT customerid,
         jour,
         salesorderid,
```


© 2026 iAlexMG. Tous droits réservés.

```
       SUM(rupture) OVER (PARTITION BY customerid ORDER BY jour
                          ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS session
FROM   marque
WHERE  customerid = 29825
ORDER  BY jour;
```

La soustraction de deux `date` rend un entier de jours sur PostgreSQL, d'où le seuil écrit `> 120` sans intervalle. Faites varier le seuil et regardez le nombre de sessions changer : le seuil est une décision métier, pas un paramètre technique, et il doit être défendu.

Comptez ensuite les sessions par client :

```
WITH marque AS (
    SELECT customerid, orderdate::date AS jour,
           CASE WHEN orderdate::date - LAG(orderdate::date) OVER (PARTITION BY customerid
                                                                  ORDER BY orderdate) > 120
                THEN 1 ELSE 0 END AS rupture
    FROM   sales.salesorderheader
),
sessions AS (
    SELECT customerid, jour,
           SUM(rupture) OVER (PARTITION BY customerid ORDER BY jour
                              ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS session
    FROM   marque
)
SELECT customerid, COUNT(DISTINCT session) AS nb_sessions, COUNT(*) AS nb_commandes
FROM   sessions
GROUP  BY customerid
ORDER  BY nb_sessions DESC
LIMIT  10;
```

6.2 La cohorte par mois de première commande

Théorie : `Theorie_09_SQL.md`, section 9.

```
WITH premiere AS (
    SELECT customerid, MIN(DATE_TRUNC('month', orderdate))::date AS mois_cohorte
    FROM   sales.salesorderheader
    GROUP  BY customerid
),
activite AS (
    SELECT h.customerid,
           p.mois_cohorte,
           DATE_TRUNC('month', h.orderdate)::date AS mois_actif
```


© 2026 iAlexMG. Tous droits réservés.

```
    FROM   sales.salesorderheader AS h
    JOIN   premiere AS p ON p.customerid = h.customerid
),
grille AS (
    SELECT mois_cohorte,
           (EXTRACT(YEAR  FROM mois_actif) - EXTRACT(YEAR  FROM mois_cohorte))::int * 12
         + (EXTRACT(MONTH FROM mois_actif) - EXTRACT(MONTH FROM mois_cohorte))::int AS rang,
           COUNT(DISTINCT customerid) AS clients
    FROM   activite
    GROUP  BY 1, 2
)
SELECT mois_cohorte, rang, clients,
       ROUND(100.0 * clients / MAX(clients) FILTER (WHERE rang = 0)
                                OVER (PARTITION BY mois_cohorte), 1) AS retention_pct
FROM   grille
WHERE  rang <= 6
ORDER  BY mois_cohorte, rang;
```

Le `FILTER` du Cours 06 sert ici à isoler la taille de la cohorte au rang zéro, et la fonction de fenêtrage la diffuse sur toutes les lignes de la cohorte. Deux lectures à faire.

Les rangs manquants. Une cohorte sans client actif au rang 3 n'a pas de ligne, et la courbe saute le point. Si vous voulez un zéro, il faut générer la grille des rangs et raccrocher, exactement comme à la partie 2.

Les cohortes trop jeunes. Les dernières cohortes n'ont pas eu six mois pour revenir. Leur rétention au rang 6 n'est pas nulle, elle est inconnue, et un tableau qui affiche zéro à cet endroit ment.

Partie 7 — Ce que le contraste vous a montré

Théorie : `Theorie_09_SQL.md`, section 10.

Sujet	**PostgreSQL, ce labo**	**L'entrepôt, `Labo_09_OLAP_SQL.md`**
Où vit l'historique	deux tables dédiées, `productlistpricehistory` et `productcosthistory`	replié dans `dbo.DimProduct`, une ligne par version du produit
Qualité des intervalles	saine, zéro chevauchement, zéro incohérence	**cassée**, la fin précède le début de plusieurs années

© 2026 iAlexMG. Tous droits réservés.

Sujet	PostgreSQL, ce labo	L'entrepôt, `Labo_09_OLAP_SQL.md`
L'écriture par intervalle fermé	perd 85 696 lignes sur 121 317, à cause des `enddate` nulles	ne renvoie rien du tout pour les versions historiques
L'écriture canonique	`LEFT JOIN LATERAL … ORDER BY startdate DESC LIMIT 1`	la même, plus `ASOF LEFT JOIN` en natif
Générer un axe de dates	`generate_series`	`dbo.DimDate`, déjà présente et déjà complète
Cadre en durée	`RANGE BETWEEN INTERVAL … PRECEDING`, plus le mode `GROUPS`	`RANGE BETWEEN INTERVAL … PRECEDING` aussi ; le labo passe quand même par l'axe et `ROWS`, la forme qui voyage
Médiane	`PERCENTILE_CONT … WITHIN GROUP`, forme longue obligatoire	idem, plus `MEDIAN`
Écart entre deux dates	une soustraction de `date` rend un entier de jours	une soustraction aussi, ou `DATEDIFF('unite', a, b)`
La journée d'un changement d'heure	la soustraction rend `1 day 01:00:00`	la soustraction rend `1 day` ; `DATE_DIFF('hour', …)` dit 25
Une période de largeur libre	`DATE_BIN`, origine obligatoire, pas de mois	`TIME_BUCKET`, origine implicite au lundi 3 janvier 2000, mois acceptés

La leçon de fond dépasse la syntaxe, et c'est celle qu'il faut emporter en entretien. Le monde transactionnel confie l'historique à une table dont c'est le seul travail, et une contrainte y veille puisque la définition de `productlistpricehistory` exige que la fin suive le début. L'entrepôt replie l'historique dans la dimension au moment de l'alimentation, et ce repliement est un traitement, donc un endroit où des erreurs se logent sans que personne les voie.

L'écriture qui s'appuie sur la seule date de début est insensible aux deux situations. C'est rarement le cas d'une écriture : celle-ci mérite d'être connue par cœur.

Nettoyage

```sql
DROP SCHEMA labo09 CASCADE;
```

Et ensuite ?

© 2026 iAlexMG. Tous droits réservés.

Le `LATERAL` a servi trois fois maintenant : le top N par groupe au Cours 07, l'aplatissement d'un tableau JSON au Cours 08, et l'as-of join ici. Trois usages, une seule idée, celle d'une sous-requête autorisée à lire la ligne de la table écrite à sa gauche.

Vous avez aussi rencontré trois fautes silencieuses en trois modules : le `NOT IN` sur colonne nullable, la valeur `JSON` comparée sans la flèche double, et l'intervalle de validité qu'on n'a pas contrôlé. La famille est maintenant complète, et vous devriez la reconnaître de loin.

La partie A s'achève ici. Le Cours 10 ouvre la partie B et pose une question que ce labo a laissée de côté : combien coûte, à la machine, l'as-of join que vous venez d'écrire, et comment le mesurer.

Prochaine étape : les exercices, dans `Exercices_corriges_09_SQL.md`.

© 2026 iAlexMG. Tous droits réservés.